ICT Class 9 - Formative Assessment 1

Formative Assessment (FA-1) - Answer Key

- Class 9

Class: Class 9

Assessment Type: Formative Assessment (FA-1)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Python Fundamentals - Review and Advanced Concepts

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b) `[2, 4]`
   - **Explanation:** `x[1::2]` starts at index 1 and steps by 2, giving elements at indices 1 and 3: `[2, 4]`.
2. **Answer:** b) `_score`
   - **Explanation:** Variable names can start with an underscore. They cannot start with a digit (`2nd_place`), contain hyphens (`total-marks`), or be reserved keywords (`class`).
3. **Answer:** b) `<class 'int'>`
   - **Explanation:** The `//` operator performs floor division, which returns an integer. `10 // 3 = 3`.
4. **Answer:** b) `*`
   - **Explanation:** The `*` operator is used for string repetition (e.g., `"ha" * 3` gives `"hahaha"`).
5. **Answer:** b) `False`
   - **Explanation:** An empty string `""` is falsy in Python, so `bool("")` returns `False`.

Section II: Short Answer (5 x 2 = 10 marks)

1. **Differentiate between `is` and `==` operators in Python.**
   - **Answer:** `==` checks if two variables have the same **value**, while `is` checks if they refer to the same **object in memory**.
   - **Marking:** 1 mark for explaining `==` compares values, 1 mark for explaining `is` compares identity. Example: `a = [1,2]; b = [1,2]; a == b` is `True`, but `a is b` is `False`.
2. **Write the output of the swap code.**
   - **Answer:** Output: `10 5`
   - **Explanation:** Python supports simultaneous assignment. `a, b = b, a` swaps the values of `a` and `b` without needing a temporary variable.
   - **Marking:** 1 mark for correct output, 1 mark for explanation of tuple unpacking.
3. **Explain the difference between `while` loop and `for` loop.**
   - **Answer:** A `for` loop iterates over a sequence (list, range, string) and is used when the number of iterations is known. A `while` loop repeats as long as a condition is `True` and is used when the number of iterations is unknown.
   - **Marking:** 1 mark for `for` loop description, 1 mark for `while` loop description with use case.
4. **Write a Python program to check if a number is a palindrome.**
   - **Answer:**

```
def is_palindrome(n):
    original = n
    reversed_num = 0
    while n > 0:
        reversed_num = reversed_num * 10 + n % 10
        n //= 10
    return original == reversed_num

print(is_palindrome(121))  # True
print(is_palindrome(123))  # False
```

- **Marking:** 1 mark for correct logic, 1 mark for working code with test cases.

5. **Purpose of `break`, `continue`, and `pass` statements.**
   - **Answer:**
     - `break`: Terminates the loop entirely. Example: Exit loop when a condition is met.
     - `continue`: Skips the current iteration and moves to the next. Example: Skip even numbers in a loop.
     - `pass`: Does nothing; acts as a placeholder. Example: Used when a statement is syntactically required.
   - **Marking:** 1 mark for any two correct explanations with examples, 1 mark for the third.

Section III: Long Answer (5 x 1 = 5 marks)

1. **Write a Python program to filter prime numbers from a list.**
   - **Answer:**

```
def is_prime(n):
    """Check if a number is prime."""
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def filter_primes(numbers):
    """Return a list of prime numbers from the input list."""
    return [n for n in numbers if is_prime(n)]

# Test cases
print(filter_primes([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]))
# Output: [2, 3, 5, 7]

print(filter_primes([11, 12, 13, 14, 15]))
# Output: [11, 13]

print(filter_primes([20, 21, 22, 23, 24, 25]))
# Output: [23]
```

   - **Marking:** 1 mark for `is_prime` function, 1 mark for `filter_primes` function, 1 mark for docstrings, 1 mark for at least 3 test cases, 1 mark for correct output and code style.

Part B: Hands-on Practical Lab Task (5 marks)

- **Answer:**

```
def count_words(sentence):
    return len(sentence.split())

def count_vowels(sentence):
    vowels = "aeiouAEIOU"
    return sum(1 for ch in sentence if ch in vowels)

def count_consonants(sentence):
    vowels = "aeiouAEIOU"
    return sum(1 for ch in sentence if ch.isalpha() and ch not in vowels)

def count_digits(sentence):
    return sum(1 for ch in sentence if ch.isdigit())

def count_special(sentence):
    return sum(1 for ch in sentence if not ch.isalnum() and ch.isspace() == False)

def vowel_frequency(sentence):
    freq = {}
    for ch in sentence.lower():
        if ch in "aeiou":
            freq[ch] = freq.get(ch, 0) + 1
    return freq

def is_palindrome_sentence(sentence):
    cleaned = ''.join(ch.lower() for ch in sentence if ch.isalnum())
    return cleaned == cleaned[::-1]

# Main
sentence = input("Enter a sentence: ")
print(f"Words: {count_words(sentence)}")
print(f"Vowels: {count_vowels(sentence)}")
print(f"Consonants: {count_consonants(sentence)}")
print(f"Digits: {count_digits(sentence)}")
print(f"Special characters: {count_special(sentence)}")
print(f"Vowel frequency: {vowel_frequency(sentence)}")
print(f"Is palindrome: {is_palindrome_sentence(sentence)}")
```

- **Marking:** 1 mark for functions for each counting operation, 1 mark for vowel frequency, 1 mark for palindrome check ignoring spaces/case, 1 mark for correct logic, 1 mark for clean code structure.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)

ICT Class 9 - Formative Assessment 2

Formative Assessment (FA-2) - Answer Key

- Class 9

Class: Class 9

Assessment Type: Formative Assessment (FA-2)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Data Structures - Lists, Tuples, Dictionaries, Sets

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b) 3
   - **Explanation:** Sets automatically remove duplicates. `{1, 2, 3, 2, 1}` becomes `{1, 2, 3}`, so `len(s)` returns 3.
2. **Answer:** c) `items()`
   - **Explanation:** `dict.items()` returns a view object displaying a list of key-value tuples. `keys()` returns only keys, `values()` returns only values.
3. **Answer:** b) `[2, 4, 6]`
   - **Explanation:** The list comprehension iterates through each row and each element, filtering even numbers (`x % 2 == 0`), producing `[2, 4, 6]`.
4. **Answer:** d) Both B and C
   - **Explanation:** Tuples can contain mixed data types and are generally faster than lists due to immutability (fixed memory).
5. **Answer:** a) `{'a': 0, 'b': 0, 'c': 0}`
   - **Explanation:** `dict.fromkeys()` creates a dictionary with the given keys and the same default value for all keys.

Section II: Short Answer (5 x 2 = 10 marks)

1. **Differentiate between a list and a tuple.**
   - **Answer:** Lists are mutable (can be modified after creation) while tuples are immutable (cannot be modified). Lists use square brackets `[]`, tuples use parentheses `()`.
   - **Marking:** 1 mark for mutability difference, 1 mark for two real-world use cases where tuples are preferred (e.g., coordinates, RGB colours, function return values).
2. **Write the output of the dictionary update code.**
   - **Answer:** Output: `{'x': 1, 'y': 3, 'z': 4}`
   - **Explanation:** `update()` merges `d2` into `d1`. Since `"y"` exists in both, the value from `d2` (3) overwrites the value in `d1` (2).
   - **Marking:** 1 mark for correct output, 1 mark for explanation of overwrite behaviour.
3. **Explain the difference between `remove()`, `pop()`, and `del`.**
   - **Answer:**
     - `remove(x)`: Removes the first occurrence of `x` from the list. Returns nothing. Raises `ValueError` if not found.

    - `pop(index)`: Removes and returns the element at the given index. Default is the last element.
    - `del`: A keyword that removes an element by index or deletes the entire variable. E.g., `del lst[0]`.
  - **Marking:** 1 mark for any two correct explanations with examples, 1 mark for the third.

4. **Write a Python program to find intersection and union of two sets.**
  - **Answer:**

```python
def intersection(set1, set2):
    result = set()
    for item in set1:
        if item in set2:
            result.add(item)
    return result

def union(set1, set2):
    result = set1.copy()
    for item in set2:
        if item not in result:
            result.add(item)
    return result

a = {1, 2, 3, 4, 5}
b = {4, 5, 6, 7, 8}

print("Intersection:", intersection(a, b))  # {4, 5}
print("Union:", union(a, b))  # {1, 2, 3, 4, 5, 6, 7, 8}
```

  - **Marking:** 1 mark for intersection logic, 1 mark for union logic without using `&` or `|` operators.

5. **What is list comprehension? Write one to generate Pythagorean triplets.**
  - **Answer:** List comprehension is a concise way to create lists using a single line of code with the syntax `[expression for item in iterable if condition]`.
  - **Marking:** 1 mark for definition, 1 mark for correct list comprehension:

```python
triplets = [(a, b, c) for a in range(1, 21) for b in range(a, 21) for c in range(b, 21) if a**2 + b**2 == c**2]
print(triplets)
# Output: [(3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15)]
```

Section III: Long Answer (5 x 1 = 5 marks)

1. **Write a Python program implementing a word frequency counter.**
  - **Answer:**

```python
import string

def word_frequency_counter(text):
    """Count word frequency in a paragraph."""
    # Remove punctuation and convert to lowercase
    cleaned = text.translate(str.maketrans('', '', string.punctuation)).lower()
    words = cleaned.split()
```

```
    # Count frequencies
    freq = {}
    for word in words:
        freq[word] = freq.get(word, 0) + 1

    # Sort by frequency descending
    sorted_words = sorted(freq.items(), key=lambda x: x[1], reverse=True)

    # Display top 5
    print("Top 5 most frequent words:")
    for word, count in sorted_words[:5]:
        print(f"  {word}: {count}")

    # Words appearing exactly once
    print("\nWords appearing exactly once:")
    for word, count in sorted_words:
        if count == 1:
            print(f"  {word}")

# Test
text = """Python is great and Python is easy. Python is used for data analysis.
Data analysis is fun. Analysis of data requires Python."""
word_frequency_counter(text)
```

- **Marking:** 1 mark for reading and cleaning text, 1 mark for counting frequencies, 1 mark for sorting and displaying top 5, 1 mark for finding unique words, 1 mark for code structure and test case.

Part B: Hands-on Practical Lab Task (5 marks)

- **Answer:**

```
students = [
    {"name": "Amit", "roll_no": 1, "marks": 85},
    {"name": "Priya", "roll_no": 2, "marks": 92},
    {"name": "Rahul", "roll_no": 3, "marks": 78},
    {"name": "Sneha", "roll_no": 4, "marks": 88},
    {"name": "Vikram", "roll_no": 5, "marks": 65},
    {"name": "Ananya", "roll_no": 6, "marks": 95},
    {"name": "Deepak", "roll_no": 7, "marks": 72},
    {"name": "Meera", "roll_no": 8, "marks": 81},
    {"name": "Arjun", "roll_no": 9, "marks": 59},
    {"name": "Kavya", "roll_no": 10, "marks": 88},
    {"name": "Rohan", "roll_no": 11, "marks": 76},
    {"name": "Divya", "roll_no": 12, "marks": 91},
    {"name": "Suresh", "roll_no": 13, "marks": 68},
    {"name": "Pooja", "roll_no": 14, "marks": 83},
    {"name": "Manish", "roll_no": 15, "marks": 70},
]

def find_topper(students):
    return max(students, key=lambda s: s["marks"])
```

```
def class_average(students):
    return sum(s["marks"] for s in students) / len(students)

def below_average(students, avg):
    return [s for s in students if s["marks"] < avg]

def sort_by_marks(students):
    return sorted(students, key=lambda s: s["marks"], reverse=True)

def display_table(students):
    print(f"{'Name':<12} {'Roll No':<10} {'Marks':<8}")
    print("-" * 32)
    for s in students:
        print(f"{s['name']:<12} {s['roll_no']:<10} {s['marks']:<8}")

# Results
topper = find_topper(students)
avg = class_average(students)
below_avg = below_average(students, avg)
sorted_students = sort_by_marks(students)

print("=== Class Topper ===")
print(f"{topper['name']} - {topper['marks']} marks\n")

print(f"=== Class Average: {avg:.2f} ===\n")

print("=== Below Average Students ===")
display_table(below_avg)

print("\n=== Students Ranked by Marks ===")
display_table(sorted_students)
```

- **Marking:** 1 mark for creating student records, 1 mark for topper function, 1 mark for average and below-average functions, 1 mark for sorting, 1 mark for formatted display.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)

ICT Class 9 - Formative Assessment 3

Formative Assessment (FA-3) - Answer Key

- Class 9

Class: Class 9

Assessment Type: Formative Assessment (FA-3)

Total Marks: 30

Duration: 90 minutes

Topics Covered: SQL - Database Management

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b) `JOIN`
   - **Explanation:** The `JOIN` clause is used to combine rows from two or more tables based on a related column. `UNION` combines results of two SELECT statements, not rows from tables.
2. **Answer:** b) Counts only distinct departments
   - **Explanation:** `COUNT(DISTINCT department)` counts only unique (non-duplicate) department values, ignoring NULLs.
3. **Answer:** c) `ALTER`
   - **Explanation:** `ALTER` is a DDL (Data Definition Language) command used to modify table structure. `SELECT`, `INSERT`, and `UPDATE` are DML commands.
4. **Answer:** b) Pattern matching
   - **Explanation:** The `LIKE` operator is used for pattern matching in SQL using wildcards `%` (any characters) and _ (single character).
5. **Answer:** d) Both B and C
   - **Explanation:** `COUNT(column_name)` and `SUM(column_name)` both ignore NULL values. `COUNT(*)` counts all rows including NULLs.

Section II: Short Answer (5 x 2 = 10 marks)

1. **Write a SQL query to create a table `Library`.**
   - **Answer:**

```sql
CREATE TABLE Library (
    book_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(50) NOT NULL,
    genre VARCHAR(30),
    price DECIMAL(10, 2),
    available BOOLEAN DEFAULT TRUE
);
```

   - **Marking:** 1 mark for correct column definitions and data types, 1 mark for PRIMARY KEY, AUTO_INCREMENT, and DEFAULT constraints.
2. **Differentiate between `WHERE` and `HAVING` clauses.**
   - **Answer:** `WHERE` filters rows before grouping (cannot use aggregate functions). `HAVING` filters groups after `GROUP BY` (can use aggregate functions).

- **Marking:** 1 mark for explaining `WHERE` with example, 1 mark for explaining `HAVING` with example.
- **Examples:**

```
SELECT department, AVG(salary) FROM Employees WHERE age > 25 GROUP BY department
HAVING AVG(salary) > 50000;
```

3. **Write SQL queries for `Students(roll, name, class, section, marks)`.**
   - **Answer:**

```
-- Students whose name starts with 'A'
SELECT * FROM Students WHERE name LIKE 'A%';

-- Students with marks between 50 and 80
SELECT * FROM Students WHERE marks BETWEEN 50 AND 80;

-- Students in sections 'A' or 'B'
SELECT * FROM Students WHERE section IN ('A', 'B');

-- Count students in each class
SELECT class, COUNT(*) AS student_count FROM Students GROUP BY class;
```

   - **Marking:** 0.5 marks per correct query.
4. **What is a Foreign Key?**
   - **Answer:** A Foreign Key is a column in one table that references the Primary Key of another table. It maintains referential integrity by ensuring that values in the foreign key column must exist in the referenced table or be NULL.
   - **Marking:** 1 mark for definition, 1 mark for example showing referential integrity (e.g., `Results.roll_no` references `Students.roll_no`).
5. **Write a SQL query to display the second highest salary.**
   - **Answer:**

```
SELECT MAX(salary) AS second_highest
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

   - **Marking:** 1 mark for correct subquery approach, 1 mark for explanation of logic (finding max of salaries less than the maximum).

Section III: Long Answer (5 x 1 = 5 marks)

1. **Write SQL queries for the given schema.**
   - **Answer:**

```
-- Percentage of each student
SELECT s.roll_no, s.name,
    ROUND(SUM(m.marks_obtained) * 100.0 / SUM(sub.max_marks), 2) AS percentage
FROM Students s
JOIN Marks m ON s.roll_no = m.roll_no
JOIN Subjects sub ON m.sub_id = sub.sub_id
GROUP BY s.roll_no, s.name;

-- Subject with highest average score
SELECT sub.sub_name, ROUND(AVG(m.marks_obtained), 2) AS avg_score
```

```
FROM Subjects sub
JOIN Marks m ON sub.sub_id = m.sub_id
GROUP BY sub.sub_id, sub.sub_name
ORDER BY avg_score DESC
LIMIT 1;

-- Students who scored above 90% in any subject
SELECT DISTINCT s.name, sub.sub_name, m.marks_obtained, sub.max_marks
FROM Students s
JOIN Marks m ON s.roll_no = m.roll_no
JOIN Subjects sub ON m.sub_id = sub.sub_id
WHERE (m.marks_obtained * 100.0 / sub.max_marks) > 90;

-- Class topper
SELECT s.class, s.name, SUM(m.marks_obtained) AS total_marks
FROM Students s
JOIN Marks m ON s.roll_no = m.roll_no
GROUP BY s.class, s.name
ORDER BY total_marks DESC
LIMIT 1;

-- Rank list by total marks
SELECT s.roll_no, s.name, s.class,
    SUM(m.marks_obtained) AS total_marks,
    RANK() OVER (ORDER BY SUM(m.marks_obtained) DESC) AS rank_position
FROM Students s
JOIN Marks m ON s.roll_no = m.roll_no
GROUP BY s.roll_no, s.name, s.class
ORDER BY rank_position;
```

- **Marking:** 1 mark per correct query (5 queries = 5 marks).

Part B: Hands-on Practical Lab Task (5 marks)

- **Answer:**

```
import sqlite3

# Connect to database
conn = sqlite3.connect('school.db')
cursor = conn.cursor()

# Create tables
cursor.execute('''CREATE TABLE IF NOT EXISTS School (
    roll INTEGER PRIMARY KEY,
    name TEXT,
    class TEXT,
    section TEXT
)''')

cursor.execute('''CREATE TABLE IF NOT EXISTS Results (
    roll INTEGER,
```

```
    sub1 INTEGER, sub2 INTEGER, sub3 INTEGER, sub4 INTEGER, sub5 INTEGER,
    FOREIGN KEY (roll) REFERENCES School(roll)
)''')

# Insert data
students = [
    (1, 'Amit', '9', 'A', 85, 78, 92, 88, 76),
    (2, 'Priya', '9', 'B', 92, 85, 88, 95, 90),
    (3, 'Rahul', '9', 'A', 68, 72, 65, 70, 58),
    (4, 'Sneha', '9', 'B', 88, 91, 85, 82, 87),
    (5, 'Vikram', '9', 'A', 55, 48, 62, 50, 45),
    (6, 'Ananya', '9', 'B', 95, 92, 98, 90, 94),
    (7, 'Deepak', '9', 'A', 72, 68, 75, 70, 65),
    (8, 'Meera', '9', 'B', 81, 84, 79, 86, 82),
]

for s in students:
    cursor.execute('INSERT OR IGNORE INTO School VALUES (?, ?, ?, ?)', (s[0], s[1], s[2],
s[3]))
    cursor.execute('INSERT OR IGNORE INTO Results VALUES (?, ?, ?, ?, ?, ?)', (s[0],
s[4], s[5], s[6], s[7], s[8]))

conn.commit()

# Average marks per student
print("=== Average Marks per Student ===")
cursor.execute('''SELECT s.name, ROUND(AVG(r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0, 2) as
avg
    FROM School s JOIN Results r ON s.roll = r.roll GROUP BY s.roll''')
for row in cursor.fetchall():
    print(f"{row[0]}: {row[1]}")

# Subject with highest class average
print("\n=== Subject with Highest Class Average ===")
cursor.execute('''SELECT 'Sub1' as subject, AVG(sub1) as avg FROM Results
    UNION ALL SELECT 'Sub2', AVG(sub2) FROM Results
    UNION ALL SELECT 'Sub3', AVG(sub3) FROM Results
    UNION ALL SELECT 'Sub4', AVG(sub4) FROM Results
    UNION ALL SELECT 'Sub5', AVG(sub5) FROM Results
    ORDER BY avg DESC LIMIT 1''')
row = cursor.fetchone()
print(f"{row[0]}: {row[1]:.2f}")

# Students needing improvement
print("\n=== Students Needing Improvement (Avg < 60%) ===")
cursor.execute('''SELECT s.name, ROUND((r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0, 2) as
avg
    FROM School s JOIN Results r ON s.roll = r.roll
    WHERE (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 < 60''')
for row in cursor.fetchall():
    print(f"{row[0]}: {row[1]}")
```

```
# Grade assignment
print("\n=== Grade Assignment ===")
cursor.execute('''SELECT s.name, (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 as avg,
    CASE
        WHEN (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 >= 90 THEN 'A'
        WHEN (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 >= 75 THEN 'B'
        WHEN (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 >= 60 THEN 'C'
        WHEN (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 >= 45 THEN 'D'
        ELSE 'F'
    END as grade
    FROM School s JOIN Results r ON s.roll = r.roll''')
for row in cursor.fetchall():
    print(f"{row[0]}: {row[1]:.2f} -> {row[2]}")

conn.close()
```

- **Marking:** 1 mark for database connection and table creation, 1 mark for inserting data, 1 mark for average calculation, 1 mark for subject with highest average, 1 mark for grade assignment logic.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)

ICT Class 9 - Formative Assessment 4

Formative Assessment (FA-4) - Answer Key

- Class 9

Class: Class 9

Assessment Type: Formative Assessment (FA-4)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Web Technologies - HTML, CSS, JavaScript

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b) `<nav>`
   - **Explanation:** The `<nav>` HTML5 tag is used to define a section containing navigation links.
2. **Answer:** c) `display: flex`
   - **Explanation:** `display: flex` creates a flexible layout container, enabling flexible alignment and distribution of child elements.
3. **Answer:** b) Changes the HTML content of element
   - **Explanation:** `innerHTML` sets or returns the HTML content (inner markup) of an element.
4. **Answer:** c) `onclick`
   - **Explanation:** The `onclick` event occurs when the user clicks on an HTML element. `onmouseover` triggers on hover, `onchange` on form element changes.
5. **Answer:** b) `rem`
   - **Explanation:** `rem` (root em) is relative to the font-size of the root `<html>` element. `em` is relative to the parent element's font-size.

Section II: Short Answer (5 x 2 = 10 marks)

1. **Differentiate between inline, internal, and external CSS.**
   - **Answer:**
     - **Inline CSS:** Applied directly to an element using the `style` attribute. Used for single-element styling.
     - **Internal CSS:** Defined in a `<style>` tag within the `<head>` section. Used for single-page styling.
     - **External CSS:** Defined in a separate `.css` file linked via `<link>` tag. Used for multi-page styling and better organisation.
   - **Marking:** 1 mark for any two correct descriptions with use cases, 1 mark for the third.
2. **Write HTML and CSS for a responsive navigation bar structure.**
   - **Answer:**

```
<nav class="navbar">
    <div class="logo">School</div>
    <ul class="nav-links">
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Contact</a></li>
    </ul>
```

```
    <div class="hamburger">&#9776;</div>
</nav>
.navbar { display: flex; justify-content: space-between; align-items: center; padding:
1rem; }
.nav-links { display: flex; list-style: none; gap: 2rem; }
.hamburger { display: none; font-size: 1.5rem; }
@media (max-width: 768px) {
    .nav-links { display: none; }
    .hamburger { display: block; }
}
```

- **Marking:** 1 mark for HTML structure with nav element, 1 mark for CSS flexbox and media query.

3. **Explain the CSS box model and `box-sizing: border-box`.**
   - **Answer:** The CSS box model consists of content, padding, border, and margin (from innermost to outermost). `box-sizing: border-box` changes the default behaviour so that padding and border are included in the element's total width and height, rather than being added on top.
   - **Marking:** 1 mark for explaining the four components, 1 mark for explaining `border-box` behaviour.
4. **Write JavaScript to validate a form field.**
   - **Answer:**

```
function validateForm() {
    var field = document.getElementById("name").value;
    if (field.trim() === "") {
        alert("Error: Field cannot be empty!");
        return false;
    }
    alert("Valid! Value: " + field);
    return true;
}
```

   - **Marking:** 1 mark for checking empty field, 1 mark for displaying alert with value or error message.
5. **Difference between `var`, `let`, and `const`.**
   - **Answer:**
     - `var`: Function-scoped, can be redeclared and updated. Hoisted.
     - `let`: Block-scoped, can be updated but not redeclared in same scope.
     - `const`: Block-scoped, cannot be updated or redeclared. Must be initialized.
   - **Marking:** 1 mark for any two correct descriptions with examples, 1 mark for the third.

Section III: Long Answer (5 x 1 = 5 marks)

1. **Create a Student Management System web page.**
   - **Answer:**

```
<!DOCTYPE html>
<html>
<head>
    <style>
```

```
        body { font-family: Arial; background: #f4f4f4; padding: 20px; }
        .form-container { background: white; padding: 20px; border-radius: 8px; max-
width: 500px; margin: auto; }
        input { width: 100%; padding: 10px; margin: 8px 0; border: 1px solid #ddd;
border-radius: 4px; box-sizing: border-box; }
        button { background: #007bff; color: white; padding: 10px 20px; border: none;
border-radius: 4px; cursor: pointer; }
        .result-card { background: #e8f5e9; padding: 15px; border-radius: 8px; margin-
top: 15px; display: none; }
    </style>
</head>
<body>
    <div class="form-container">
        <h2>Student Report Card</h2>
        <input type="text" id="name" placeholder="Student Name">
        <input type="text" id="class" placeholder="Class">
        <input type="text" id="section" placeholder="Section">
        <input type="number" id="roll" placeholder="Roll Number">
        <input type="number" id="sub1" placeholder="Subject 1 Marks">
        <input type="number" id="sub2" placeholder="Subject 2 Marks">
        <input type="number" id="sub3" placeholder="Subject 3 Marks">
        <button onclick="calculate()">Calculate</button>
        <div class="result-card" id="result"></div>
    </div>
    <script>
        function calculate() {
            var s1 = parseFloat(document.getElementById("sub1").value);
            var s2 = parseFloat(document.getElementById("sub2").value);
            var s3 = parseFloat(document.getElementById("sub3").value);
            if (isNaN(s1) || isNaN(s2) || isNaN(s3) || s1 < 0 || s1 > 100 || s2 < 0 ||
s2 > 100 || s3 < 0 || s3 > 100) {
                alert("Please enter valid marks (0-100)");
                return;
            }
            var total = s1 + s2 + s3;
            var percentage = (total / 300 * 100).toFixed(2);
            var grade = percentage >= 90 ? "A" : percentage >= 75 ? "B" : percentage
>= 60 ? "C" : percentage >= 45 ? "D" : "F";
            var resultDiv = document.getElementById("result");
            resultDiv.style.display = "block";
            resultDiv.innerHTML = "<h3>Result</h3><p>Total: " + total + "</
p><p>Percentage: " + percentage + "%</p><p>Grade: " + grade + "</p>";
        }
    </script>
</body>
</html>
```

- **Marking:** 1 mark for HTML form with all fields, 1 mark for CSS styling, 1 mark for calculate function, 1 mark for grade logic and result display, 1 mark for input validation.

Part B: Hands-on Practical Lab Task (5 marks)

- **Answer:**

```
<!DOCTYPE html>
<html>
<head>
    <style>
        body { font-family: Arial; text-align: center; background: #1a1a2e; color:
white; }
        .quiz-container { max-width: 600px; margin: 50px auto; padding: 20px; }
        .progress-bar { background: #333; height: 20px; border-radius: 10px; margin: 20px
0; }
        .progress { background: #00b4d8; height: 100%; border-radius: 10px; transition:
width 0.3s; }
        .options button { display: block; width: 100%; padding: 12px; margin: 8px 0;
border: 2px solid #00b4d8; background: transparent; color: white; border-radius: 5px;
cursor: pointer; font-size: 16px; transition: all 0.3s; }
        .options button:hover { background: #00b4d8; }
        .correct { background: #2ecc71 !important; border-color: #2ecc71; }
        .incorrect { background: #e74c3c !important; border-color: #e74c3c; }
    </style>
</head>
<body>
    <div class="quiz-container">
        <h1>Quiz App</h1>
        <div class="progress-bar"><div class="progress" id="progress"></div></div>
        <div id="question"></div>
        <div class="options" id="options"></div>
        <p id="score"></p>
    </div>
    <script>
        var questions = [
            { q: "What is 2+2?", options: ["3","4","5","6"], answer: 1 },
            { q: "Capital of France?", options: ["London","Berlin","Paris","Rome"],
answer: 2 },
            { q: "Largest planet?", options: ["Earth","Mars","Jupiter","Saturn"], answer:
2 }
        ];
        var current = 0, score = 0;
        function loadQuestion() {
            if (current >= questions.length) {
                document.getElementById("question").innerHTML = "<h2>Quiz Complete!</
h2>";
                document.getElementById("options").innerHTML = "";
                document.getElementById("score").innerHTML = "Score: " + score + "/" +
questions.length;
                return;
            }
            var q = questions[current];
            document.getElementById("question").innerHTML = "<h3>" + q.q + "</h3>";
            document.getElementById("progress").style.width = ((current+1)/
```

```
questions.length*100) + "%";
            var html = "";
            q.options.forEach(function(opt, i) {
                html += "<button onclick='check(" + i + ")'>" + opt + "</button>";
            });
            document.getElementById("options").innerHTML = html;
        }
        function check(selected) {
            var btns = document.querySelectorAll(".options button");
            btns.forEach(function(b, i) {
                b.disabled = true;
                if (i === questions[current].answer) b.classList.add("correct");
                if (i === selected && i !== questions[current].answer)
b.classList.add("incorrect");
            });
            if (selected === questions[current].answer) score++;
            current++;
            setTimeout(loadQuestion, 1000);
        }
        loadQuestion();
    </script>
</body>
</html>
```

- **Marking:** 1 mark for question display with options, 1 mark for score tracking and correct/ incorrect highlighting, 1 mark for progress bar, 1 mark for final score with message, 1 mark for CSS styling with animations.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)

ICT Class 9 - Practical Lab Exam 1

Practical Lab Exam 1 - Answer Key

- Class 9

Class: Class 9

Assessment Type: Practical Lab Exam 1

Total Marks: 20

Duration: 45 minutes

Topics Covered: Python Programming and Data Structures

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: Student Grade Manager (5 marks)

- **Answer:**

```
students = []

def add_student(name, roll_no, subjects):
    """Add a new student with marks in 5 subjects."""
    student = {
        "name": name,
        "roll_no": roll_no,
        "subjects": subjects
    }
    students.append(student)

def update_marks(roll_no, subject, marks):
    """Update marks for an existing student."""
    for student in students:
        if student["roll_no"] == roll_no:
            student["subjects"][subject] = marks
            return True
    return False

def calculate_grade(average):
    """Calculate grade based on average."""
    if average >= 90: return "A"
    elif average >= 75: return "B"
    elif average >= 60: return "C"
    elif average >= 45: return "D"
    else: return "F"

def get_average(student):
    """Calculate student's average."""
    marks = list(student["subjects"].values())
    return sum(marks) / len(marks)

def find_class_topper():
    """Find the student with highest average."""
    if not students: return None
    return max(students, key=lambda s: get_average(s))
```

```
def find_subject_toppers():
    """Find topper in each subject."""
    toppers = {}
    for subject in students[0]["subjects"]:
        best = max(students, key=lambda s: s["subjects"][subject])
        toppers[subject] = best["name"]
    return toppers

def display_report_card():
    """Display formatted report card for all students."""
    print(f"{'Name':<12} {'Roll':<6} {'Avg':<8} {'Grade':<6}")
    print("-" * 35)
    for s in students:
        avg = get_average(s)
        grade = calculate_grade(avg)
        print(f"{s['name']:<12} {s['roll_no']:<6} {avg:<8.2f} {grade:<6}")

# Test data
add_student("Amit", 101, {"Math": 85, "Science": 78, "English": 92, "Hindi": 88, "CS":
76})
add_student("Priya", 102, {"Math": 92, "Science": 85, "English": 88, "Hindi": 95, "CS":
90})
add_student("Rahul", 103, {"Math": 68, "Science": 72, "English": 65, "Hindi": 70, "CS":
58})

display_report_card()
print(f"\nClass Topper: {find_class_topper()['name']}")
print(f"Subject Toppers: {find_subject_toppers()}")
```

- **Marking:**
  - 1 mark for student data structure (list of dictionaries)
  - 1 mark for add and update functions
  - 1 mark for grade calculation logic
  - 1 mark for topper and subject topper functions
  - 1 mark for formatted report card display

Task 2: Inventory Management System (5 marks)

- **Answer:**

```
inventory = []

def add_item(item_id, name, quantity, price, category):
    """Add new item to inventory."""
    inventory.append({
        "item_id": item_id,
        "name": name,
        "quantity": quantity,
        "price": price,
        "category": category
    })
```

```
def update_stock(item_id, amount):
    """Increase (positive) or decrease (negative) stock."""
    for item in inventory:
        if item["item_id"] == item_id:
            item["quantity"] += amount
            return True
    return False

def search_items(query):
    """Search by name or category."""
    return [i for i in inventory if query.lower() in i["name"].lower()
            or query.lower() in i["category"].lower()]

def total_inventory_value():
    """Calculate total value of all inventory."""
    return sum(i["quantity"] * i["price"] for i in inventory)

def sort_by(field):
    """Sort inventory by price or quantity."""
    return sorted(inventory, key=lambda x: x[field])

def low_stock_alert(threshold=5):
    """Items with quantity below threshold."""
    return [i for i in inventory if i["quantity"] < threshold]

def export_to_file(filename):
    """Export inventory data to a text file."""
    with open(filename, 'w') as f:
        for item in inventory:
            f.write(f"{item['item_id']},{item['name']},{item['quantity']},"
                    f"{item['price']},{item['category']}\n")
    print(f"Exported to {filename}")

# Menu-driven interface
def menu():
    while True:
        print("\n=== Inventory Management ===")
        print("1. Add Item  2. Update Stock  3. Search")
        print("4. Total Value  5. Sort  6. Low Stock  7. Export  8. Exit")
        choice = input("Enter choice: ")
        if choice == "1":
            add_item(input("ID: "), input("Name: "), int(input("Qty: ")),
                     float(input("Price: ")), input("Category: "))
        elif choice == "3":
            items = search_items(input("Search: "))
            for i in items: print(i)
        elif choice == "7":
            export_to_file("inventory.txt")
        elif choice == "8":
            break
```

```
# Test
add_item("P001", "Pen", 100, 10, "Stationery")
add_item("P002", "Notebook", 50, 50, "Stationery")
add_item("L001", "Laptop", 5, 45000, "Electronics")
print("Total Value:", total_inventory_value())
print("Low Stock:", low_stock_alert())
```

- **Marking:**
  - 1 mark for inventory data structure
  - 1 mark for add and update stock functions
  - 1 mark for search and sort functions
  - 1 mark for total value and low stock alert
  - 1 mark for file export function

Task 3: Pattern Generator (5 marks)

- **Answer:**

```
def pattern_a(rows):
    """Right-angled triangle of stars."""
    for i in range(1, rows + 1):
        print("*" * i)

def pattern_b(rows):
    """Pyramid pattern."""
    for i in range(1, rows + 1):
        print(" " * (rows - i) + "*" * (2 * i - 1))

def pattern_c(rows):
    """Number pattern."""
    for i in range(1, rows + 1):
        print(str(i) * i)

def display_pattern(choice, rows):
    """Display the chosen pattern."""
    if choice == "A":
        pattern_a(rows)
    elif choice == "B":
        pattern_b(rows)
    elif choice == "C":
        pattern_c(rows)

# Test with 5 rows
print("Pattern A:")
display_pattern("A", 5)

print("\nPattern B:")
display_pattern("B", 5)

print("\nPattern C:")
display_pattern("C", 5)
```

- **Output:**

```
Pattern A:
*
**
***
****
*****

Pattern B:
    *
   ***
  *****
 *******
*********

Pattern C:
1
22
333
4444
55555
```

- **Marking:**
  - 1 mark for Pattern A (right triangle)
  - 1 mark for Pattern B (pyramid with spaces)
  - 1 mark for Pattern C (number repetition)
  - 1 mark for function structure with choice parameter
  - 1 mark for custom row input support and clean code

Viva Voce (5 marks)

1. **Difference between function definition and function call.** (1 mark)
   - **Answer:** A function definition declares the function with `def`, specifying parameters and body. A function call executes the function using its name followed by parentheses with arguments.
2. **Mutable vs immutable data types.** (1 mark)
   - **Answer:** Mutable (list, dict, set) can be changed after creation. Immutable (int, float, string, tuple, frozenset) cannot be changed; operations create new objects.
3. **Shallow copy vs deep copy of a nested list.** (1 mark)
   - **Answer:** Shallow copy creates a new outer list but inner lists are shared references. Deep copy creates completely independent copies of all nested objects. `copy.deepcopy()` is used for deep copy.
4. **Purpose of `__init__` method in a Python class.** (1 mark)
   - **Answer:** `__init__` is the constructor method that initialises object attributes when an object is created. Yes, an object can be created without it using default behaviour.
5. **Linear search vs binary search.** (1 mark)

- **Answer:** Linear search checks each element sequentially (O(n)), works on unsorted lists. Binary search divides sorted array in half each step (O(log n)), requires sorted list.

ICT Class 9 - Practical Lab Exam 2

Practical Lab Exam 2 - Answer Key

- Class 9

Class: Class 9

Assessment Type: Practical Lab Exam 2

Total Marks: 20

Duration: 45 minutes

Topics Covered: SQL with Python and Web Development

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: SQL Database Operations with Python (5 marks)

- **Answer:**

```
import sqlite3

conn = sqlite3.connect('school.db')
cursor = conn.cursor()

# Create tables
cursor.execute('''CREATE TABLE IF NOT EXISTS Students (
    roll_no INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    class TEXT,
    section TEXT,
    admission_date DATE
)''')

cursor.execute('''CREATE TABLE IF NOT EXISTS Subjects (
    sub_id INTEGER PRIMARY KEY,
    sub_name TEXT NOT NULL,
    max_marks INTEGER
)''')

cursor.execute('''CREATE TABLE IF NOT EXISTS Results (
    roll_no INTEGER,
    sub_id INTEGER,
    marks_obtained INTEGER,
    FOREIGN KEY (roll_no) REFERENCES Students(roll_no),
    FOREIGN KEY (sub_id) REFERENCES Subjects(sub_id)
)''')

# Insert data
students = [
    (1, 'Amit', '9', 'A', '2024-04-01'),
    (2, 'Priya', '9', 'B', '2024-04-01'),
    (3, 'Rahul', '9', 'A', '2024-04-02'),
    (4, 'Sneha', '9', 'B', '2024-04-02'),
    (5, 'Vikram', '9', 'A', '2024-04-03'),
    (6, 'Ananya', '9', 'B', '2024-04-03'),
]
```

```
cursor.executemany('INSERT OR IGNORE INTO Students VALUES (?, ?, ?, ?, ?)', students)

subjects = [(1, 'Mathematics', 100), (2, 'Science', 100), (3, 'English', 100), (4,
'Hindi', 100)]
cursor.executemany('INSERT OR IGNORE INTO Subjects VALUES (?, ?, ?)', subjects)

results = [
    (1, 1, 85), (1, 2, 78), (1, 3, 92), (1, 4, 88),
    (2, 1, 92), (2, 2, 85), (2, 3, 88), (2, 4, 95),
    (3, 1, 68), (3, 2, 72), (3, 3, 65), (3, 4, 70),
    (4, 1, 88), (4, 2, 91), (4, 3, 85), (4, 4, 82),
    (5, 1, 55), (5, 2, 48), (5, 3, 62), (5, 4, 50),
    (6, 1, 95), (6, 2, 92), (6, 3, 98), (6, 4, 90),
]
cursor.executemany('INSERT OR IGNORE INTO Results VALUES (?, ?, ?)', results)
conn.commit()

# Students with marks above 80
print("=== Students with marks above 80 ===")
cursor.execute('''SELECT s.name, sub.sub_name, r.marks_obtained
    FROM Students s JOIN Results r ON s.roll_no = r.roll_no
    JOIN Subjects sub ON r.sub_id = sub.sub_id
    WHERE r.marks_obtained > 80''')
for row in cursor.fetchall():
    print(f"{row[0]} - {row[1]}: {row[2]}")

# Subject-wise average and highest
print("\n=== Subject-wise Statistics ===")
cursor.execute('''SELECT sub.sub_name, ROUND(AVG(r.marks_obtained),2),
MAX(r.marks_obtained)
    FROM Subjects sub JOIN Results r ON sub.sub_id = r.sub_id
    GROUP BY sub.sub_id''')
for row in cursor.fetchall():
    print(f"{row[0]}: Avg={row[1]}, Highest={row[2]}")

# Students ranked by total marks
print("\n=== Students Ranked by Total ===")
cursor.execute('''SELECT s.name, SUM(r.marks_obtained) as total
    FROM Students s JOIN Results r ON s.roll_no = r.roll_no
    GROUP BY s.roll_no ORDER BY total DESC''')
for i, row in enumerate(cursor.fetchall(), 1):
    print(f"{i}. {row[0]}: {row[1]}")

# Update and compare
print("\n=== Before/After Update ===")
cursor.execute('SELECT marks_obtained FROM Results WHERE roll_no=3 AND sub_id=1')
old = cursor.fetchone()[0]
cursor.execute('UPDATE Results SET marks_obtained=75 WHERE roll_no=3 AND sub_id=1')
conn.commit()
cursor.execute('SELECT marks_obtained FROM Results WHERE roll_no=3 AND sub_id=1')
new = cursor.fetchone()[0]
```

```
print(f"Rahul Math: {old} -> {new}")

conn.close()
```

- **Marking:**
  - 1 mark for database connection and table creation with foreign keys
  - 1 mark for inserting student, subject, and result data
  - 1 mark for query: students with marks above 80
  - 1 mark for subject-wise statistics and ranking
  - 1 mark for update with before/after comparison

Task 2: HTML and CSS Web Page (5 marks)

- **Answer:**

```
<!-- school_portal.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>School Portal</title>
    <link rel="stylesheet" href="portal.css">
</head>
<body>
    <header class="navbar">
        <div class="logo">our school School</div>
        <nav class="nav-links">
            <a href="#">Home</a>
            <a href="#">About</a>
            <a href="#">Academics</a>
            <a href="#">Contact</a>
        </nav>
    </header>

    <section class="hero">
        <h1>Welcome to our school School</h1>
        <p>Empowering students through quality education</p>
        <button class="cta-btn">Learn More</button>
    </section>

    <section class="students-section">
        <h2>Student Results</h2>
        <table class="student-table">
            <thead>
                <tr><th>Name</th><th>Class</th><th>Section</th><th>Marks</th><th>Grade</
th></tr>
            </thead>
            <tbody>
                <tr><td>Amit</td><td>9</td><td>A</td><td>85</td><td>B</td></tr>
                <tr><td>Priya</td><td>9</td><td>B</td><td>92</td><td>A</td></tr>
                <tr><td>Rahul</td><td>9</td><td>A</td><td>72</td><td>C</td></tr>
```

```
                <tr><td>Sneha</td><td>9</td><td>B</td><td>88</td><td>B</td></tr>
                <tr><td>Vikram</td><td>9</td><td>A</td><td>58</td><td>D</td></tr>
                <tr><td>Ananya</td><td>9</td><td>B</td><td>95</td><td>A</td></tr>
            </tbody>
        </table>
    </section>

    <section class="contact-section">
        <h2>Contact Us</h2>
        <form class="contact-form">
            <input type="text" placeholder="Name" required>
            <input type="email" placeholder="Email" required>
            <input type="text" placeholder="Subject" required>
            <textarea placeholder="Message" rows="4" required></textarea>
            <button type="submit">Submit</button>
        </form>
    </section>

    <footer>
        <p>&copy; 2026 our school School. All rights reserved.</p>
    </footer>
</body>
</html>
/* portal.css */
* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: Arial, sans-serif; }

.navbar { display: flex; justify-content: space-between; align-items: center; padding:
1rem 5%; background: #2c3e50; color: white; }
.logo { font-size: 1.5rem; font-weight: bold; }
.nav-links a { color: white; text-decoration: none; margin-left: 2rem; }
.nav-links a:hover { text-decoration: underline; }

.hero { text-align: center; padding: 4rem 2rem; background: #3498db; color: white; }
.hero h1 { font-size: 2.5rem; margin-bottom: 1rem; }
.cta-btn { padding: 12px 30px; background: #e74c3c; color: white; border: none; border-
radius: 5px; font-size: 1rem; cursor: pointer; }

.students-section { padding: 2rem 5%; }
.student-table { width: 100%; border-collapse: collapse; margin-top: 1rem; }
.student-table th, .student-table td { padding: 12px; border: 1px solid #ddd; text-align:
left; }
.student-table thead { background: #2c3e50; color: white; }
.student-table tbody tr:nth-child(even) { background: #f2f2f2; }
.student-table tbody tr:hover { background: #ddd; }

.contact-section { padding: 2rem 5%; background: #ecf0f1; }
.contact-form { max-width: 500px; display: flex; flex-direction: column; gap: 10px;
margin-top: 1rem; }
.contact-form input, .contact-form textarea { padding: 10px; border: 1px solid #bbb;
border-radius: 4px; }
```

```
.contact-form button { padding: 12px; background: #27ae60; color: white; border: none;
border-radius: 4px; cursor: pointer; }

footer { text-align: center; padding: 1rem; background: #2c3e50; color: white; }
```

- **Marking:**
  - 1 mark for header with navigation links
  - 1 mark for hero section with CTA button
  - 1 mark for table with zebra striping and hover effect
  - 1 mark for contact form with all required fields
  - 1 mark for CSS using Flexbox and responsive styling

Task 3: Interactive Web Application (5 marks)

- **Answer:**

```
<!-- todo_app.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Todo App</title>
    <style>
        body { font-family: Arial; max-width: 600px; margin: 50px auto; background:
#f5f5f5; }
        .todo-container { background: white; padding: 20px; border-radius: 8px; box-
shadow: 0 2px 10px rgba(0,0,0,0.1); }
        h1 { text-align: center; color: #333; }
        .input-section { display: flex; gap: 10px; margin-bottom: 20px; }
        .input-section input, .input-section select { padding: 10px; border: 1px solid
#ddd; border-radius: 4px; }
        .input-section button { padding: 10px 20px; background: #3498db; color: white;
border: none; border-radius: 4px; cursor: pointer; }
        .task-list { list-style: none; }
        .task-item { display: flex; align-items: center; padding: 12px; margin: 8px 0;
border-radius: 4px; background: #f9f9f9; transition: all 0.3s; }
        .task-item:hover { transform: translateX(5px); }
        .task-item.completed span { text-decoration: line-through; color: #999; }
        .priority-high { border-left: 4px solid #e74c3c; }
        .priority-medium { border-left: 4px solid #f39c12; }
        .priority-low { border-left: 4px solid #27ae60; }
        .task-item input[type="checkbox"] { margin-right: 10px; }
        .task-item span { flex: 1; }
        .delete-btn { background: #e74c3c; color: white; border: none; padding: 5px 10px;
border-radius: 3px; cursor: pointer; }
        .summary { margin-top: 20px; padding: 15px; background: #ecf0f1; border-radius:
4px; text-align: center; }
    </style>
</head>
<body>
    <div class="todo-container">
        <h1>Todo List</h1>
```

```
        <div class="input-section">
            <input type="text" id="taskInput" placeholder="Enter task">
            <select id="prioritySelect">
                <option value="high">High</option>
                <option value="medium">Medium</option>
                <option value="low">Low</option>
            </select>
            <button onclick="addTask()">Add</button>
        </div>
        <ul class="task-list" id="taskList"></ul>
        <div class="summary" id="summary"></div>
    </div>
    <script>
        var tasks = [];
        function addTask() {
            var title = document.getElementById("taskInput").value.trim();
            var priority = document.getElementById("prioritySelect").value;
            if (!title) return alert("Enter a task!");
            tasks.push({ title: title, priority: priority, completed: false });
            document.getElementById("taskInput").value = "";
            render();
        }
        function toggleTask(index) {
            tasks[index].completed = !tasks[index].completed;
            render();
        }
        function deleteTask(index) {
            tasks.splice(index, 1);
            render();
        }
        function render() {
            var html = "";
            tasks.forEach(function(task, i) {
                var cls = "task-item priority-" + task.priority + (task.completed ? "
completed" : "");
                html += "<li class='" + cls + "'>" +
                    "<input type='checkbox'" + (task.completed ? " checked" : "") + "
onclick='toggleTask(" + i + ")'>" +
                    "<span>" + task.title + " [" + task.priority + "]</span>" +
                    "<button class='delete-btn' onclick='deleteTask(" + i + ")'>X</
button></li>";
            });
            document.getElementById("taskList").innerHTML = html;
            var total = tasks.length;
            var completed = tasks.filter(function(t) { return t.completed; }).length;
            document.getElementById("summary").innerHTML =
                "Total: " + total + " | Completed: " + completed + " | Pending: " +
(total - completed);
        }
        render();
    </script>
```

```
</body>
</html>
```

- **Marking:**
  - 1 mark for task input with priority selection
  - 1 mark for colour-coded priority indicators
  - 1 mark for toggle complete (strikethrough) and delete
  - 1 mark for summary display (total/completed/pending)
  - 1 mark for CSS styling with smooth transitions

Viva Voce (5 marks)

1. **Difference between INNER JOIN and LEFT JOIN.** (1 mark)
   - **Answer:** INNER JOIN returns only rows with matching values in both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table (NULL for non-matching right side).

   ```
   SELECT s.name, r.marks FROM Students s INNER JOIN Results r ON s.roll = r.roll; -- Only students with results
   SELECT s.name, r.marks FROM Students s LEFT JOIN Results r ON s.roll = r.roll; -- All students
   ```

2. **How does CSS specificity work?** (1 mark)
   - **Answer:** CSS specificity determines which rule takes priority: Inline styles (highest) > Internal/Embedded styles > External stylesheets > Browser defaults (lowest). If two rules have the same specificity, the last one defined wins.

   ```
   p { color: blue; }          /* External - lowest */
   <style>p { color: red; }</style>  /* Internal - medium */
   <p style="color: green;">    /* Inline - highest */
   ```

3. **Purpose of a Foreign Key in SQL.** (1 mark)
   - **Answer:** A Foreign Key creates a link between two tables by referencing the Primary Key of another table. It maintains referential integrity by ensuring that values in the foreign key column must exist in the referenced table or be NULL.

   ```
   CREATE TABLE Results (
       roll_no INTEGER,
       FOREIGN KEY (roll_no) REFERENCES Students(roll_no)
   );
   ```

4. **Difference between WHERE and HAVING.** (1 mark)
   - **Answer:** `WHERE` filters individual rows before grouping (cannot use aggregate functions). `HAVING` filters groups after `GROUP BY` (can use aggregate functions like COUNT, AVG, SUM).

   ```
   SELECT class, AVG(marks) FROM Students WHERE marks > 50 GROUP BY class HAVING AVG(marks) > 70;
   ```

5. **Responsive design vs adaptive design.** (1 mark)
   - **Answer:**
     - Responsive design: Fluid layout that adapts to any screen size using relative units and media queries. Example: CSS Flexbox with percentage widths.

- Adaptive design: Uses fixed layouts for specific screen sizes (breakpoints). Example: Serving different HTML/CSS for mobile vs desktop.

ICT Class 9 - Summative Assessment 1

Summative Assessment (SA-1) - Answer Key

- Class 9

Class: Class 9

Assessment Type: Summative Assessment (SA-1)

Total Marks: 40

Duration: 2 hours

Topics Covered: Python Fundamentals, Data Structures, SQL, Web Technologies

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: MCQ (10 x 1 = 10 marks)

1. **Answer:** a) `[1, 3, 5, 2, 4]`
   - **Explanation:** `x[::2]` gives elements at indices 0,2,4: `[1,3,5]`. `x[1::2]` gives indices 1,3: `[2,4]`. Concatenation gives `[1,3,5,2,4]`.
2. **Answer:** a) `dict = {"a": 1, "b": 2}`
   - **Explanation:** Valid dictionary creation uses curly braces with key-value pairs. Option B creates a list of tuples. Option C uses a list as key (invalid). Option D is invalid syntax.
3. **Answer:** b) `"hon"`
   - **Explanation:** `s[-3:]` takes the last 3 characters from “Python”: “hon”.
4. **Answer:** c) `DROP TABLE`
   - **Explanation:** `DROP TABLE` removes the table structure and all data permanently. `DELETE` removes rows, `TRUNCATE` removes all rows but keeps structure.
5. **Answer:** b) `<ul>`
   - **Explanation:** `<ul>` defines an unordered (bulleted) list. `<ol>` is for ordered lists, `<li>` defines list items.
6. **Answer:** b) O(log n)
   - **Explanation:** Binary search repeatedly halves the search space, giving O(log n) time complexity on a sorted list.
7. **Answer:** b) `font-family`
   - **Explanation:** `font-family` CSS property specifies the font for an element. E.g., `font-family: Arial, sans-serif;`.
8. **Answer:** b) `[1, 2, 3]`
   - **Explanation:** `d.values()` returns the dictionary values. `list()` converts them to a list: `[1, 2, 3]`.
9. **Answer:** b) `document.getElementById()`
   - **Explanation:** `document.getElementById()` selects a single element by its unique ID attribute.
10. **Answer:** b) Counts all rows including NULL values
    - **Explanation:** `COUNT(*)` counts every row in the table, including those with NULL values.

Part B: Short Answer (8 x 2 = 16 marks)

1. **Write a function `merge_dicts(d1, d2)` that merges dictionaries, adding values for common keys.**
   - **Answer:**

```python
def merge_dicts(d1, d2):
    result = d1.copy()
    for key, value in d2.items():
        result[key] = result.get(key, 0) + value
    return result

print(merge_dicts({"a": 1, "b": 2}, {"b": 3, "c": 4}))
# Output: {'a': 1, 'b': 5, 'c': 4}
```

   - **Marking:** 1 mark for function logic, 1 mark for correct output with test case.
2. **Write the output of the diagonal extraction code.**
   - **Answer:** Output: `[1, 5, 9]`
   - **Explanation:** `lst[i][i]` extracts diagonal elements: `lst[0][0]=1`, `lst[1][1]=5`, `lst[2][2]=9`.
   - **Marking:** 1 mark for correct output, 1 mark for explanation of diagonal index logic.
3. **Write SQL queries for `Employees(emp_id, name, department, salary, joining_date)`.**
   - **Answer:**

```sql
-- Employees who joined in the last 2 years
SELECT * FROM Employees WHERE joining_date >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR);

-- Second highest salary
SELECT MAX(salary) FROM Employees WHERE salary < (SELECT MAX(salary) FROM Employees);

-- Departments where average salary > 50000
SELECT department, AVG(salary) AS avg_salary
FROM Employees GROUP BY department HAVING AVG(salary) > 50000;
```

   - **Marking:** 1 mark per query (3 queries = partial, 1.5 marks each rounded to 2).
4. **Differentiate between DELETE, TRUNCATE, and DROP.**
   - **Answer:**
     - `DELETE`: DML command, removes specific rows with WHERE clause, can be rolled back.
     - `TRUNCATE`: DDL command, removes all rows, faster than DELETE, cannot be rolled back in most DBs.
     - `DROP`: DDL command, removes the entire table structure and data permanently.
   - **Marking:** 1 mark for any two correct distinctions, 1 mark for examples.
5. **Explain CSS Flexbox and create a three-column layout.**
   - **Answer:**

```css
.container {
    display: flex;
}
.side {
    flex: 1;
}
.center {
```

```
    flex: 2;
}
```

- **Explanation:** Flexbox is a one-dimensional layout model. `flex: 1` gives equal width to side columns, `flex: 2` gives the center column double width.
- **Marking:** 1 mark for explaining Flexbox, 1 mark for correct CSS code.

6. **What is responsive web design? Explain three techniques.**
   - **Answer:** Responsive web design ensures web pages render well on all devices. Three techniques:
     1. Media queries: Apply different CSS based on screen size.
     2. Flexible grids: Use relative units (%, vw) instead of fixed pixels.
     3. Flexible images: Use `max-width: 100%` to prevent overflow.
   - **Marking:** 1 mark for definition, 1 mark for any three techniques with brief explanation.
7. **Write a Python program to implement a stack using a list.**
   - **Answer:**

```python
stack = []

def push(item):
    stack.append(item)

def pop():
    if not is_empty():
        return stack.pop()
    return "Stack is empty"

def peek():
    if not is_empty():
        return stack[-1]
    return "Stack is empty"

def is_empty():
    return len(stack) == 0

def display():
    print("Stack:", stack)

push(10)
push(20)
push(30)
display()       # [10, 20, 30]
print(peek())   # 30
print(pop())    # 30
display()       # [10, 20]
```

   - **Marking:** 1 mark for stack operations, 1 mark for proper implementation with test.
8. **Write a SQL query to find the nth highest salary (n=5).**
   - **Answer:**

```sql
SELECT DISTINCT salary
FROM Employees
```

```
ORDER BY salary DESC
LIMIT 1 OFFSET 4;
```

- Alternative using subqueries:

```
SELECT MAX(salary) FROM Employees
WHERE salary NOT IN (
    SELECT DISTINCT salary FROM Employees
    ORDER BY salary DESC LIMIT 4
);
```

- **Marking:** 1 mark for LIMIT/OFFSET approach, 1 mark for subquery approach or explanation.

Part C: Long Answer (4 x 3 = 12 marks)

1. **Write a Library Management System program.**
   - **Answer:** Key components:
     - Dictionary to store books: `{"isbn": {"title": ..., "author": ..., "copies": ..., "issued": []}}`
     - Functions: `add_book()`, `issue_book()`, `return_book()`, `search_book()`, `calculate_fine()`
     - Error handling for invalid ISBN, unavailable books, overdue returns
     - Fine calculation: Rs 5 per day overdue
   - **Marking:** 1 mark for data structure design, 1 mark for core functions, 1 mark for error handling and fine calculation.
2. **Write SQL queries for the E-Commerce Database.**
   - **Answer:**

```
-- Top 5 customers by spending
SELECT c.name, SUM(o.total_amount) AS total_spent
FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
GROUP BY c.cust_id, c.name ORDER BY total_spent DESC LIMIT 5;

-- Products never ordered
SELECT p.name FROM Products p
WHERE p.prod_id NOT IN (SELECT DISTINCT prod_id FROM Order_Items);

-- Monthly revenue
SELECT MONTH(order_date) AS month, SUM(total_amount) AS revenue
FROM Orders WHERE YEAR(order_date) = YEAR(CURDATE())
GROUP BY MONTH(order_date) ORDER BY month;

-- Most popular category
SELECT p.category, SUM(oi.quantity) AS total_qty
FROM Order_Items oi JOIN Products p ON oi.prod_id = p.prod_id
GROUP BY p.category ORDER BY total_qty DESC LIMIT 1;

-- Customers with more than 3 different products
SELECT c.name, COUNT(DISTINCT oi.prod_id) AS products_ordered
FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
```

```
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY c.cust_id, c.name HAVING COUNT(DISTINCT oi.prod_id) > 3;
```

- **Marking:** 1 mark for any three correct queries, 2 marks for all five correct.

3. **Create a Student Report Card Generator web page.**
   - **Answer:** Key components:
     - HTML form with all required fields using CSS Grid layout
     - JavaScript validation (marks 0-100)
     - Grade calculation (A:90+, B:75-89, C:60-74, D:45-59, F:below 45)
     - Styled report card display below form
     - Print button using `window.print()`
   - **Marking:** 1 mark for HTML form and CSS Grid, 1 mark for JavaScript logic, 1 mark for report card display and print.
4. **Write a Contact Book program using file handling.**
   - **Answer:** Key components:
     - Dictionary of lists for contacts storage
     - File operations: `json.dump()` / `json.load()` for persistence
     - Operations: add, search (partial match), update, delete, display
     - Search by name, phone, or email
     - Sort alphabetically or by date
     - CSV export using `csv` module
   - **Marking:** 1 mark for data structure and file handling, 1 mark for CRUD operations, 1 mark for search, sort, and export features.

Part D: Application Based (2 x 1 = 2 marks)

1. **Student Attendance Tracking System.**
   - **Answer:**
     - Python: Dictionary with date keys, list of student names for present/absent.
     - SQL: Tables `Students(roll, name)`, `Attendance(roll, date, status)` with foreign key.
     - Web Interface: HTML table for daily view, JavaScript to filter by date/student, CSS for styling.
   - **Marking:** 1 mark for Python and SQL components, 1 mark for web interface components.
2. **Online Bookstore Inventory System.**
   - **Answer:**
     - SQL: `Books(isbn, title, author, price, stock)` with proper constraints.
     - Python: Read CSV with `csv.reader()`, validate data, insert into database.
     - Web Form: HTML form to add books, JavaScript validation, POST to server.
     - Common Query: `SELECT * FROM Books WHERE stock < 5` for low stock alert.
   - **Marking:** 1 mark for SQL schema and common query, 1 mark for Python and web components.

ICT Class 9 - Summative Assessment 2

Summative Assessment (SA-2) - Answer Key

- Class 9

Class: Class 9

Assessment Type: Summative Assessment (SA-2)

Total Marks: 40

Duration: 2 hours

Topics Covered: Algorithms, Cyber Security, Emerging Technologies, Computational Thinking

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: MCQ (10 x 1 = 10 marks)

1. **Answer:** c) $O(n^2)$
   - **Explanation:** Bubble sort compares adjacent elements and swaps them, resulting in $O(n^2)$ time complexity in the worst case.
2. **Answer:** c) Ambiguity
   - **Explanation:** A good algorithm must be unambiguous. Ambiguity violates the definiteness characteristic of algorithms.
3. **Answer:** b) A social engineering attack to steal credentials
   - **Explanation:** Phishing uses deceptive emails/websites to trick users into revealing passwords, credit card details, or other sensitive information.
4. **Answer:** c) Blockchain
   - **Explanation:** Blockchain is the underlying technology of Bitcoin. It provides a decentralised, immutable ledger for transactions.
5. **Answer:** b) To filter network traffic and block threats
   - **Explanation:** A firewall monitors incoming and outgoing network traffic and blocks malicious traffic based on security rules.
6. **Answer:** c) Parallelogram
   - **Explanation:** In flowcharts, a parallelogram represents input/output operations. Rectangle = process, Diamond = decision, Oval = start/end.
7. **Answer:** b) An email asking for password pretending to be from a bank
   - **Explanation:** This is a classic example of phishing - impersonating a trusted entity to steal credentials.
8. **Answer:** b) A subset of AI that enables systems to learn from data
   - **Explanation:** Machine Learning allows computers to learn patterns from data and make decisions without being explicitly programmed.
9. **Answer:** b) Symmetric encryption
   - **Explanation:** Symmetric encryption uses the same key for both encryption and decryption (e.g., AES). Asymmetric uses different keys (public/private).
10. **Answer:** b) Internet of Things
    - **Explanation:** IoT refers to the network of physical devices embedded with sensors, software, and connectivity to exchange data.

Part B: Short Answer (8 x 2 = 16 marks)

1. **Write an algorithm to find the second smallest element in an unsorted list.**
   - **Answer:**

```
ALGORITHM SecondSmallest(list)
INPUT: An unsorted list of n elements
OUTPUT: The second smallest element

1. IF n < 2 THEN RETURN "List too small"
2. SET smallest = list[0]
3. SET second = infinity
4. FOR i = 1 TO n-1 DO
    a. IF list[i] < smallest THEN
        second = smallest
        smallest = list[i]
    b. ELSE IF list[i] < second AND list[i] != smallest THEN
        second = list[i]
5. RETURN second
```

   - Time complexity: O(n) - single pass through the list.
   - **Marking:** 1 mark for algorithm, 1 mark for time complexity analysis.
2. **Differentiate between encryption and decryption.**
   - **Answer:**
     - Encryption: Converting plaintext into ciphertext using an algorithm and key. Example: Encrypting a message before sending via email.
     - Decryption: Converting ciphertext back into plaintext using the appropriate key. Example: Receiving and reading the encrypted email.
   - **Marking:** 1 mark for each definition with example.
3. **Explain three types of malware.**
   - **Answer:**
     - Virus: Attaches to legitimate files, spreads when executed. Prevention: Use antivirus software.
     - Worm: Self-replicating, spreads across networks without user interaction. Prevention: Keep software updated, use firewalls.
     - Ransomware: Encrypts files and demands payment for decryption. Prevention: Regular backups, avoid suspicious links.
   - **Marking:** 1 mark for any two types with prevention, 1 mark for the third.
4. **What is Big Data? Explain the 5 V's.**
   - **Answer:** Big Data refers to extremely large datasets that cannot be processed by traditional tools.
     - Volume: Massive amount of data generated.
     - Velocity: Speed at which data is generated and processed.
     - Variety: Different formats (structured, unstructured, semi-structured).
     - Veracity: Accuracy and trustworthiness of data.
     - Value: Useful insights extracted from data.

- **Marking:** 1 mark for definition, 1 mark for any four V's correctly explained.

5. **Write a Python program to implement selection sort.**
   - **Answer:**

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

data = [64, 25, 12, 22, 11]
print("Original:", data)
print("Sorted:", selection_sort(data))
# Trace: [64,25,12,22,11] -> [11,25,12,22,64] -> [11,12,25,22,64] -> [11,12,22,25,64]
-> [11,12,22,25,64]
```

   - **Marking:** 1 mark for correct algorithm, 1 mark for trace on given input.

6. **Explain two-factor authentication.**
   - **Answer:** Two-factor authentication (2FA) requires two different forms of verification: something you know (password) and something you have (phone/OTP) or something you are (biometric). It is more secure because even if the password is compromised, the attacker still needs the second factor.
   - **Marking:** 1 mark for explanation of 2FA, 1 mark for why it's more secure.

7. **What is cloud computing? Differentiate public, private, and hybrid cloud.**
   - **Answer:** Cloud computing delivers computing services (servers, storage, databases) over the internet.
     - Public Cloud: Owned by third-party providers (e.g., AWS, Google Cloud). Shared infrastructure.
     - Private Cloud: Dedicated to a single organisation. More control and security.
     - Hybrid Cloud: Combines public and private clouds. Allows data and applications to move between them.
   - **Marking:** 1 mark for cloud computing definition, 1 mark for differentiation with examples.

8. **Write pseudocode for binary search and trace it.**
   - **Answer:**

```
ALGORITHM BinarySearch(arr, target)
INPUT: Sorted array arr, target value
OUTPUT: Index of target or -1

1. SET low = 0, high = length(arr) - 1
2. WHILE low <= high DO
    a. SET mid = (low + high) / 2
    b. IF arr[mid] == target THEN RETURN mid
    c. ELSE IF arr[mid] < target THEN low = mid + 1
```

```
    d. ELSE high = mid - 1
3. RETURN -1
```

- Trace for target 23: low=0,high=9,mid=4(16<23) -> low=5,high=9,mid=7(56>23) -> low=5,high=6,mid=5(23==23) -> Found at index 5.
- **Marking:** 1 mark for pseudocode, 1 mark for correct trace.

Part C: Long Answer (4 x 3 = 12 marks)

1. **Write a Merge Sort program and trace it.**
   - **Answer:** Key components:
     - Divide array into halves recursively until single elements
     - Merge sorted halves in correct order
     - Trace: [38,27,43,3,9,82,10] -> divide -> merge steps
     - Time complexity: O(n log n) for best, average, and worst cases
   - **Marking:** 1 mark for algorithm, 1 mark for trace, 1 mark for time complexity analysis.
2. **Cyber Safety Policy for a School.**
   - **Answer:** Must include:
     - Five threats: Phishing, Malware, Cyberbullying, Identity theft, Password attacks
     - Prevention for each: Education, software updates, reporting mechanisms, strong passwords, 2FA
     - Incident response: Identify, Contain, Eradicate, Recover, Lessons Learned
     - Monitoring: Content filters, activity logs, awareness programs
   - **Marking:** 1 mark for threats and prevention, 1 mark for incident response plan, 1 mark for monitoring approach.
3. **Explain AI, IoT, and Blockchain with examples.**
   - **Answer:**
     - AI: Systems that mimic human intelligence. Applications: Smart tutors, automated grading. Ethical concerns: Bias, privacy, job displacement.
     - IoT: Network of connected physical devices. Components: Sensors, actuators, gateway, cloud. Smart home example: Smart thermostat adjusting temperature automatically.
     - Blockchain: Distributed ledger with blocks linked by cryptography. Verification: Consensus mechanisms (proof of work/stake). Beyond crypto: Supply chain tracking, digital certificates.
   - **Marking:** 1 mark for each technology (definition, example, and one additional point).
4. **Design a Sorting Algorithm Visualiser project.**
   - **Answer:** Key components:
     - Problem statement: Visualise sorting algorithms step by step for educational purposes
     - Data structure: Array stored as list, steps stored as list of states
     - Algorithms: Bubble sort (compare adjacent), Insertion sort (shift elements), Selection sort (find minimum)
     - Web interface: JavaScript animations using `setTimeout()`, colour coding for comparisons/swaps

- **Marking:** 1 mark for problem statement and data design, 1 mark for algorithm descriptions, 1 mark for web visualisation approach.

Part D: Application Based (2 x 1 = 2 marks)

1. **Social Media Data Protection.**
   - **Answer:**
     - Encryption: End-to-end encryption for messages (e.g., Signal protocol)
     - Firewalls/IDS: Monitor server traffic, detect DDoS attacks
     - Password policies: Minimum length, complexity, 2FA enforcement
     - Regulations: GDPR, IT Act compliance, data minimisation principles
   - **Marking:** 1 mark for technical measures, 1 mark for policies and regulations.
2. **Healthcare Digitisation.**
   - **Answer:**
     - Cloud: HIPAA-compliant storage, scalable for medical records
     - IoT: Wearable devices monitoring heart rate, sending data to doctors
     - Cyber security: Encryption at rest and in transit, access controls, audit logs
     - Ethical considerations: Patient consent, data ownership, right to be forgotten
   - **Marking:** 1 mark for cloud and IoT applications, 1 mark for security and ethics.

ICT Class 9 - Unit Test 1

Unit Test 1 - Answer Key

- Class 9

Class: Class 9

Assessment Type: Unit Test 1

Total Marks: 10

Duration: 30 minutes

Topics Covered: Python Fundamentals

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b) 512
   - **Explanation:** Exponentiation is right-associative. `2 ** 3 ** 2` = `2 ** 9` = `512`.
2. **Answer:** c) `array`
   - **Explanation:** `array` is not a built-in Python data type. Python has `list`, `tuple`, `dict`, and `set`.
3. **Answer:** a) `el`
   - **Explanation:** `x[:]` copies the string "Hello". `y[1]` = "e", `y[3]` = "l", so concatenation gives "el".
4. **Answer:** b) `except`
   - **Explanation:** Python uses try/except for exception handling. catch is used in other languages like Java/C#.
5. **Answer:** b) `False`
   - **Explanation:** `not True` = `False`. `False or False and True` = `False or False` = `False`. `and` has higher precedence than `or`.

Section II: Short Answer (5 x 1 = 5 marks)

1. **Write the output of the for loop.**
   - **Answer:** Output: `1 4 7`
   - **Explanation:** `range(1, 10, 3)` generates 1, 4, 7 (stops before 10).
2. **Write a Python function `count_vowels(s)`.**
   - **Answer:**

```python
def count_vowels(s):
    vowels = "aeiouAEIOU"
    count = 0
    for ch in s:
        if ch in vowels:
            count += 1
    return count

print(count_vowels("Hello World"))  # Output: 3
```

   - **Marking:** 1 mark for function definition, 1 mark for correct call and output.
3. **Difference between `range(5)` and `range(1, 5)`.**
   - **Answer:**
     - `range(5)` generates: 0, 1, 2, 3, 4 (starts from 0 by default)

- ▸ `range(1, 5)` generates: 1, 2, 3, 4 (starts from 1, stops before 5)

4. **Write a program to check if a number is an Armstrong number.**
   - **Answer:**

```
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    total = sum(int(d) ** power for d in digits)
    return total == n

print(is_armstrong(153))  # True (1³ + 5³ + 3³ = 1 + 125 + 27 = 153)
print(is_armstrong(123))  # False
```

5. **What happens when you divide by zero in Python?**
   - **Answer:** Python raises a `ZeroDivisionError` exception. It can be handled using try/except blocks:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Viva Voce (5 marks)

1. **Difference between a function definition and a function call.** (1 mark)
   - **Answer:** A function definition declares the function using `def` with a name and body. A function call executes the defined function by using its name followed by parentheses. E.g., `def greet():` is definition; `greet()` is call.
2. **Explain mutable vs immutable data types.** (1 mark)
   - **Answer:** Mutable types (list, dict, set) can be modified after creation. Immutable types (int, float, string, tuple) cannot be modified; operations create new objects.
3. **How does Python handle shallow copy vs deep copy of a nested list?** (1 mark)
   - **Answer:** A shallow copy creates a new outer list but references the same inner objects. A deep copy creates completely independent copies of all nested objects.
4. **Purpose of `__init__` method in a Python class.** (1 mark)
   - **Answer:** `__init__` is a constructor method called automatically when an object is created. It initialises the object's attributes. Yes, an object can be created without it (uses default).
5. **Difference between linear search and binary search.** (1 mark)
   - **Answer:** Linear search checks each element sequentially (O(n)). Binary search divides the sorted array in half each step (O(log n)). Binary search requires a sorted list.

ICT Class 9 - Unit Test 2

Unit Test 2 - Answer Key

- Class 9

Class: Class 9

Assessment Type: Unit Test 2

Total Marks: 10

Duration: 30 minutes

Topics Covered: Data Structures

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b) `[1, 2, 5]`
   - **Explanation:** `d["a"]` references the list `[1, 2]`. `append(5)` adds 5 to that list, making it `[1, 2, 5]`.
2. **Answer:** b) `sort()`
   - **Explanation:** `sort()` sorts the list in place and modifies the original list. `sorted()` returns a new sorted list without modifying the original.
3. **Answer:** b) `([1, 2, 5], [3, 4])`
   - **Explanation:** Tuples are immutable, but the list inside the tuple is mutable. `t[0].append(5)` modifies the list at index 0, which is allowed.
4. **Answer:** d) Both B and C
   - **Explanation:** `list[:]` and `list.copy()` both create shallow copies of the list. Option A (`copy = list`) only creates a reference, not a copy.
5. **Answer:** a) An arbitrary element from the set
   - **Explanation:** `set.pop()` removes and returns an arbitrary element from the set. Sets are unordered, so the element returned is not predictable.

Section II: Short Answer (5 x 1 = 5 marks)

1. **Write the output of the sorted list slice.**
   - **Answer:** Output: `[9, 6, 5]`
   - **Explanation:** `sorted(nums, reverse=True)` gives `[9, 6, 5, 4, 3, 2, 1]`. `[:3]` takes the first three elements.
2. **Write a program to merge two dictionaries (summing values for common keys).**
   - **Answer:**

```python
def merge_dicts(d1, d2):
    result = d1.copy()
    for key, value in d2.items():
        if key in result:
            result[key] += value
        else:
            result[key] = value
    return result

d1 = {"a": 1, "b": 2, "c": 3}
d2 = {"b": 4, "c": 5, "d": 6}
```

```
print(merge_dicts(d1, d2))
# Output: {'a': 1, 'b': 6, 'c': 8, 'd': 6}
```

3. **What will be the output and why?**
   - **Answer:** Output: `[1, 2, 3, 4]`
   - **Explanation:** `lst2 = lst` makes both variables reference the same list object. When `lst2.append(4)` is called, it modifies the same list that `lst` points to.
4. **Differentiate between `pop()` and `remove()` for dictionaries.**
   - **Answer:**
     - `dict.pop(key)`: Removes the key-value pair and returns the value. Can provide a default value if key not found.
     - `dict.pop(key, default)`: Same as above but returns `default` if key doesn't exist (no error).
     - `del dict[key]`: Removes the key-value pair. Raises `KeyError` if key not found.
   - **Marking:** 1 mark for any two correct explanations, 1 mark for examples.
5. **Write a program to flatten a nested list.**
   - **Answer:**

```
def flatten(nested_list):
    flat = []
    for sublist in nested_list:
        for item in sublist:
            flat.append(item)
    return flat

nested = [[1, 2], [3, 4], [5, 6, 7]]
print(flatten(nested))
# Output: [1, 2, 3, 4, 5, 6, 7]
```

ICT Class 9 - Unit Test 3

Unit Test 3 - Answer Key

- Class 9

Class: Class 9

Assessment Type: Unit Test 3

Total Marks: 10

Duration: 30 minutes

Topics Covered: SQL

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b) `ALTER TABLE ... ADD`
   - **Explanation:** `ALTER TABLE table_name ADD column_name datatype` is the correct SQL command to add a new column to an existing table.
2. **Answer:** a) Top 3 students by marks
   - **Explanation:** `ORDER BY marks DESC` sorts marks from highest to lowest. `LIMIT 3` returns only the first 3 rows.
3. **Answer:** c) `ORDER()`
   - **Explanation:** `ORDER()` is not an SQL aggregate function. `SUM()`, `COUNT()`, and `AVG()` are aggregate functions.
4. **Answer:** c) Missing/unknown value
   - **Explanation:** `IS NULL` checks for NULL values, which represent missing or unknown data in SQL.
5. **Answer:** b) `GROUP BY`
   - **Explanation:** `GROUP BY` groups rows that share common values, typically used with aggregate functions like `COUNT()`, `SUM()`, `AVG()`.

Section II: Short Answer (5 x 1 = 5 marks)

1. **Write SQL queries for `Products(pid, name, price, category, stock)`.**
   - **Answer:**

```sql
-- Products with stock less than 10
SELECT * FROM Products WHERE stock < 10;

-- Count products in each category
SELECT category, COUNT(*) AS product_count FROM Products GROUP BY category;

-- Average price per category
SELECT category, ROUND(AVG(price), 2) AS avg_price FROM Products GROUP BY category;
```

2. **Differentiate between `INNER JOIN` and `LEFT JOIN`.**
   - **Answer:** `INNER JOIN` returns only rows that have matching values in both tables. `LEFT JOIN` returns all rows from the left table and matching rows from the right table (NULL for non-matching right rows).

```sql
-- INNER JOIN: Only students with results
SELECT s.name, r.marks FROM Students s INNER JOIN Results r ON s.roll = r.roll;
```

```
-- LEFT JOIN: All students, with results if available
SELECT s.name, r.marks FROM Students s LEFT JOIN Results r ON s.roll = r.roll;
```

3. **Write a query to display the third highest salary.**
   - **Answer:**

```
SELECT DISTINCT salary
FROM Employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

   - Alternative using subquery:

```
SELECT MAX(salary) FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees
    WHERE salary < (SELECT MAX(salary) FROM Employees));
```

4. **What is a Primary Key?**
   - **Answer:** A Primary Key is a column (or set of columns) that uniquely identifies each row in a table. It cannot contain NULL values and must be unique. A table can have only one PRIMARY KEY, but it can consist of multiple columns (composite primary key).
5. **Write SQL to display duplicate records from a table.**
   - **Answer:**

```
-- Create a table with duplicates
CREATE TABLE Employees_dup (id INT, name VARCHAR(50), dept VARCHAR(30));
INSERT INTO Employees_dup VALUES (1, 'Amit', 'IT'), (2, 'Priya', 'HR'), (1, 'Amit',
'IT');

-- Find duplicate records
SELECT id, name, dept, COUNT(*) as count
FROM Employees_dup
GROUP BY id, name, dept
HAVING COUNT(*) > 1;
```

ICT Class 9 - Unit Test 4

Unit Test 4 - Answer Key

- Class 9

Class: Class 9

Assessment Type: Unit Test 4

Total Marks: 10

Duration: 30 minutes

Topics Covered: Web Technologies - HTML, CSS, JavaScript

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Answer Key - Grading Guidelines

> **Note to Teacher:** This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b) `<video>`
   - **Explanation:** The `<video>` HTML5 element is used to embed video content in a web page. `<embed>` can also be used but `<video>` is the standard tag.
2. **Answer:** b) `order`
   - **Explanation:** The `order` CSS property changes the visual order of flex items within a flex container.
3. **Answer:** b) `"object"`
   - **Explanation:** `typeof null` returns `"object"` in JavaScript due to a historical bug in the language specification.
4. **Answer:** a) `push()`
   - **Explanation:** `push()` adds one or more elements to the end of an array and returns the new length.
5. **Answer:** a) `text-shadow`
   - **Explanation:** `text-shadow` CSS property adds shadow effects to text. Syntax: `text-shadow: h-offset v-offset blur color`.

Section II: Short Answer (5 x 1 = 5 marks)

1. **Write HTML code for a form with email validation.**
   - **Answer:**

```html
<form>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email"
           pattern="[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$"
           required>
    <button type="submit">Submit</button>
</form>
```

   - **Marking:** 1 mark for correct form structure, 1 mark for `pattern` attribute and `required`.
2. **Difference between `display: none` and `visibility: hidden`.**
   - **Answer:**
     - `display: none`: Removes the element entirely from the document flow. The space it occupied is reclaimed by other elements.
     - `visibility: hidden`: Hides the element but it still occupies its original space in the layout.
3. **Write JavaScript to find the largest element in an array.**

- **Answer:**

```
var arr = [23, 56, 12, 89, 45];
var max = arr[0];
for (var i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}
console.log("Largest element: " + max);  // 89
```

4. **Purpose of the `meta viewport` tag.**
   - **Answer:** The `<meta name="viewport" content="width=device-width, initial-scale=1.0">` tag tells the browser to set the width of the page to the device's screen width and set the initial zoom level. It is essential for responsive web design on mobile devices.
5. **Write CSS media queries for screens smaller than 600px.**
   - **Answer:**

```
body {
    background-color: white;
}

@media screen and (max-width: 600px) {
    body {
        background-color: lightblue;
    }
}
```